SOURCE / INTERPRETERS / CAPABILITY / PROOF

SSTI

Explained

Understand template trust, prove server-side evaluation, and build safer rendering systems

TEMPLATE SOURCE

Who controls the instructions?

RENDERED RESULT

What capability is demonstrated?

Start with templates, values, objects and trust.

Find the source. Prove the capability. Verify the repair.

Practice with a real engine and bounded dummy effects.

Haroon Shahid

First edition / Version 1.0 / 1 November 2026

Before you begin

SSTI Explained

Understand template trust, prove server-side evaluation, and build safer rendering systems

By Haroon Shahid. First edition, version 1.0. 1 November 2026. Prepared with AI assistance.

Copyright 2026 Haroon Shahid. The purchased publication is licensed for personal use under LICENSE.txt. The separately identified lab code is MIT licensed. The free preview may be shared unchanged with attribution. No resale or public redistribution of the paid publication is granted.

This book teaches responsible analysis through fictional examples and bounded local practice. It does not certify expertise. Use only systems and accounts you own or are explicitly authorized to assess.

The validation report identifies executed tests and limits. Framework, engine, and isolation guidance is version-sensitive. The local lab uses a real restricted Jinja engine, two fixed loopback listeners, and bounded dummy capabilities. It is not a production security library.

PDF code can soft-wrap for page fit. The offline HTML and source preserve original lines. Responsive HTML and scalable diagrams provide an alternative reading format; no full PDF accessibility conformance is claimed.

Free preview: introduction, roadmap and Chapters 1-3. The full package includes all 56 chapters, answered reviews, the offline reading edition, practice workbooks, reference sheets, and the local Render Forge lab.

Contents

Before you begin 2
Contents 3
Introduction 4
Learning roadmap 5
01. A template is a document with instructions 7
02. Follow a request to the machine that renders 9
03. Escaping output does not protect template source 11
Sources and further reading 13

Introduction

The same characters, a different role

A support portal asks for a name. One implementation displays the name as text. Another joins it into a template and asks a server to interpret the result. A third saves it for an export worker. The characters can be identical while their authority changes.

This book teaches that change from the beginning. You will separate source, context, output, interpreter, capability, and evidence. You will learn why HTML escaping can work while source injection remains, why a small expression does not prove arbitrary RCE, and why a restricted authoring feature can evaluate expressions without being broken.

You can start without security or programming knowledge. The first eight chapters explain templates, web requests, types, objects, parsing, trust, and classification. Later stages add discovery, controlled exploitation, engine contracts, prevention, modern systems, testing, reporting, and independent review.

Learn a method you can transfer

Find a controlling input. Follow its role through storage and transformations. Identify the real interpreter. Compare behavior against a clear permission contract. Observe the smallest relevant effect. Repair source ownership, context, capabilities, or another failed boundary. Verify legitimate behavior and state what remains unknown.

The book uses original scenarios and educational diagrams. Each chapter has an objective, analogy, primary visual, trust-view revision visual, worked explanation, takeaway, and three answered checks. The companion kit adds predictions, guided missions, a review clinic, evidence and report templates, flow models, an engine atlas, and a mastery tracker.

Practice with a real, bounded engine

Render Forge uses actual Jinja compilation and rendering. Fixed-data, weak-source, expression-context, and second-pass modes make the key transitions visible. Minimal and broad dummy contexts show how available capabilities change impact. Stored drafts, tenant checks, and manual queued snapshots teach lifecycle decisions.

The lab uses fixed local listeners and dummy effects. Its fixed file helper, local HTTP callback, dummy action, and fixed child-process marker are deliberately limited. It does not supply arbitrary network, filesystem, or command targets or reproduce every unrestricted engine and sandbox escape. Its privileged audit is a teaching view, not ordinary attacker access.

Read actively

Predict before revealing. After each mission, write source, role, interpreter, context, policy, result, consequence, and limits. Use the glossary when a term interrupts understanding. Revisit a small concept after a day, a week, and a month. Transfer the method to another authorized lab instead of memorizing only one expression.

A book cannot honestly guarantee an expert title, bug bounty, or complete future coverage. The useful goal is that you can independently explain and verify a new rendering path. Practice only on systems and accounts you own or are explicitly authorized to assess. Careful evidence is part of advanced skill.

Learning roadmap

Reader	Suggested route	Evidence to produce
Absolute beginner	Read in order; use glossary and local presets	Source/context/output map and reasoned predictions
Developer	Foundations, engine contract, Chapters 33-50	Fixed-source repair, capability inventory and regression tests
Authorized tester	Foundations, Chapters 9-32 and 49-56	Paired proof, capability ledger and bounded report
Architect	All trust stages, Chapters 33-48 and capstone	Authoring, worker, tenant, version and delivery contracts

Seven connected stages

1. **Rendering foundations, 1-8.** Explain source, values, output, types, objects, parsing and trust. Gate: show why braces and HTML interpretation alone do not establish SSTI.

2. **Finding and proof, 9-16.** Inventory document makers, use differential tests, locate evaluation, identify context and engine, and run the first repair. Gate: a reproducible paired server-evaluation proof with a normal baseline.

3. **Capability and impact, 17-24.** Distinguish data, object graphs, files, host execution, callbacks, business actions, resources and loaders. Gate: every impact claim has observed evidence and labeled prerequisites.

4. **Engine contracts, 25-32.** Compare real source, policy, helper and compilation models across engine families. Gate: explain why one copied expression or sandbox assumption cannot cover them all.

5. **Defensive design, 33-40.** Keep source controlled, minimize context, design authoring, maintain policies, bind artifacts, isolate workers and protect outputs. Gate: a working repair plus allow and deny tests.

6. **Modern systems, 41-48.** Trace documents, APIs, SSR, tenants, queues, AI prompts, cloud identities, chains and changes. Gate: a lifecycle map with each interpreter and authority decision named.

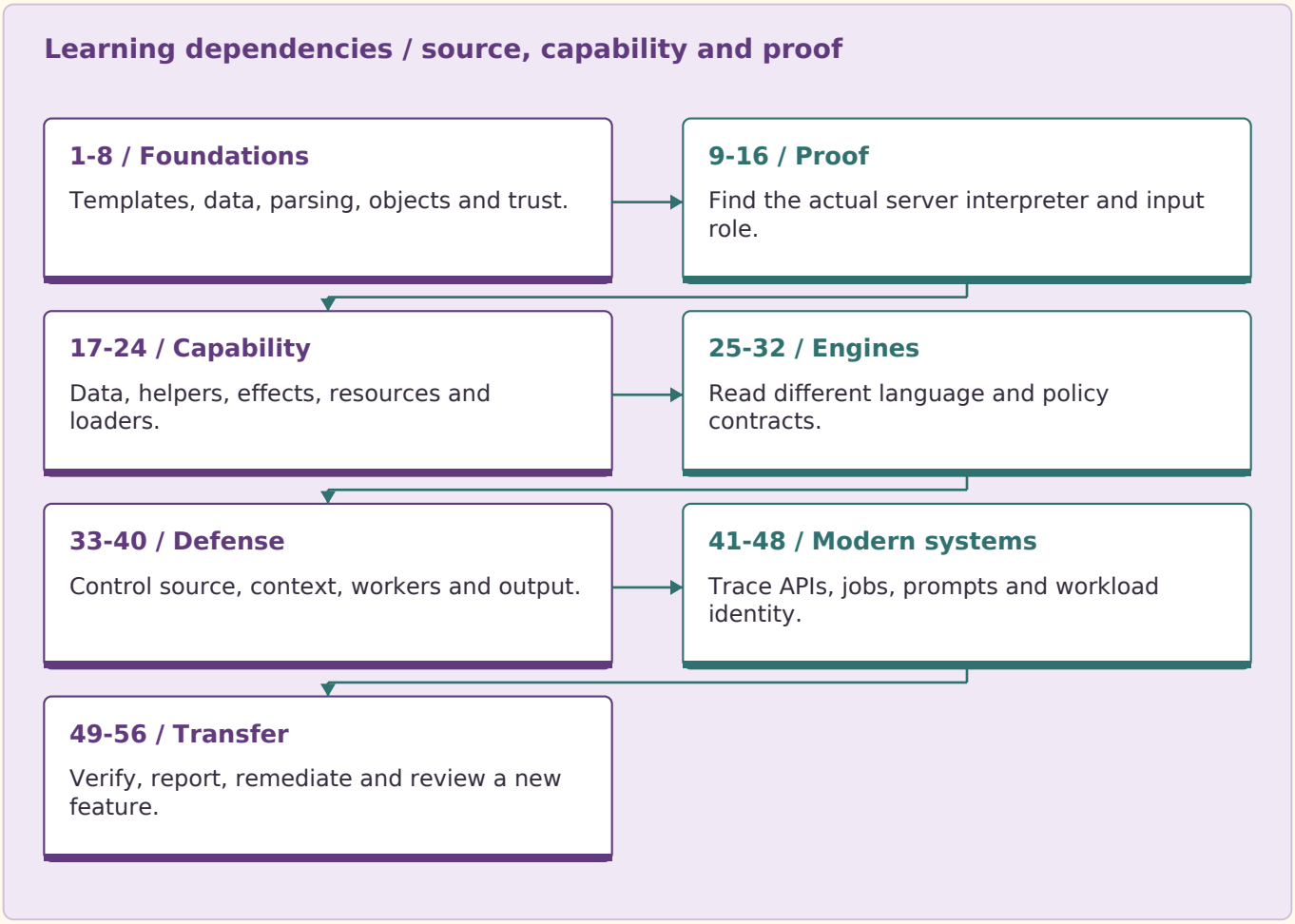
7. **Professional transfer, 49-56.** Preserve evidence, test meaningful contracts, diagnose ambiguity, read advisories, report, remediate and solve the capstone. Gate: independent review of a new authorized rendering feature.

One useful study cycle

Read one connected group. Close the book and explain the source/value difference. Predict two cases. Perform one local mission. Write one result and one limit. Revisit your explanation the next day, then after a week and a month. Use the mastery tracker to identify the next missing artifact rather than to award yourself a title.

The 40-case prediction deck trains decisions before requests. The 32 guided missions connect the method to executable local behavior. The 24 worked cases train classification, chain reasoning

and partial-fix judgment. Flow libraries and the engine atlas support review; they are not standards-complete production implementations.



Read left to right within each stage, then follow the connector to the next row. Each later stage uses the earlier mental model.

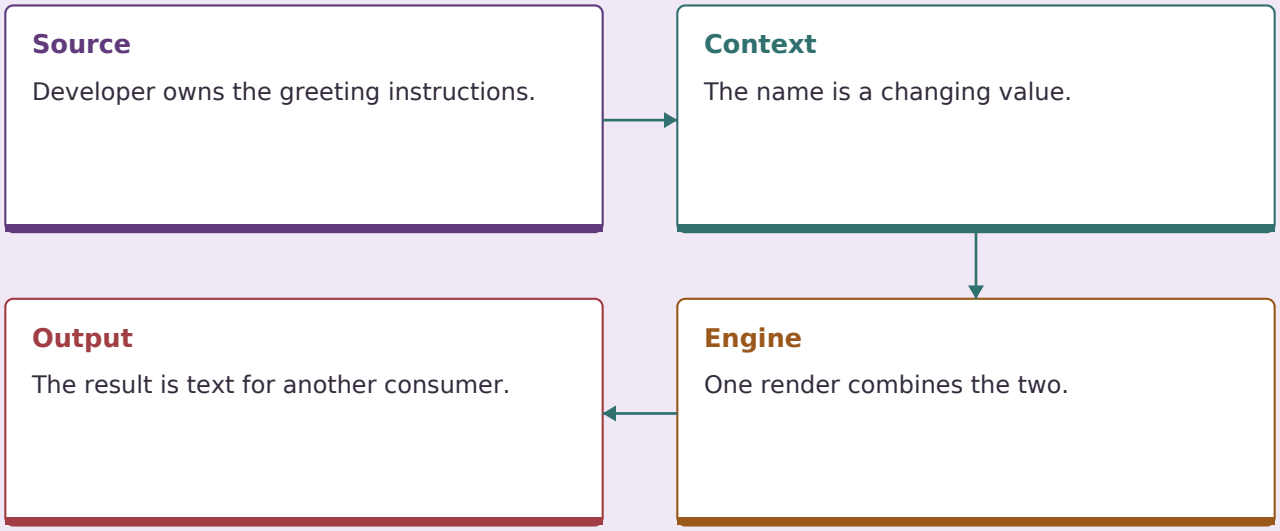
I. UNDERSTAND RENDERING BEFORE INJECTION

01. A template is a document with instructions

Learn to: Separate the reusable document, its changing values, and the program that combines them.

Think of it this way: A letter form contains a name box. The person filling the box should not be allowed to rewrite the printing instructions.

01A / Source, context, and output are different roles



Ask whether input became source, stayed data, or became output for another parser.

Start with a greeting

A template is text with places where a program can insert values. A template engine is the software that reads those instructions and produces a result. Rendering means producing that result. The result might be a web page, email, invoice, configuration file, or prompt for another system.

Here is a small Jinja template. Jinja is one particular engine, not the name of all templating.

```
Hello {{ name }}
```

The developer supplies a value for name. If name is Mira, the rendered text is Hello Mira. The two opening and two closing braces mark an expression in this engine's usual syntax. Other engines have other rules. Braces in an ordinary text file do nothing by themselves.

Three things to keep apart

The **source** is the template text that the engine reads as instructions. The **context** is the collection of values made available while rendering. The **output** is the resulting text. Keeping these three things separate is the foundation of the whole book.

Suppose name contains the literal text `{{ 13 * 17 }}`. If the application passes it as a value to the fixed greeting, a normal single render inserts that text. It does not usually treat the inserted value as a second template. The expected result contains the braces.

If the application instead builds source by joining Hello with that same input, the input now occupies the instruction area. Rendering that new source can produce 221. The visible input stayed the same; its role changed. That role change is the important security event.

Templates are useful

Without templates, developers would rebuild similar documents repeatedly. Templates separate layout from changing information. Loops can repeat invoice rows. Conditions can hide an empty address section. This is useful work, so removing all templating is rarely the right repair.

The first question is who may control each part. An ordinary customer may change a display name. A trusted developer may change the invoice source. A product may intentionally let customers author templates under a restricted contract. Those are different permissions, not just different input boxes.

Try the mental experiment

Write source, context, and output as three separate notes. Put Mira in context and predict the result. Replace Mira with brace text and predict again. Do not assume evaluation merely because the input resembles syntax. Later chapters show how to test whether a real server interpreted it.

Key takeaway. Ask whether input became source, stayed data, or became output for another parser.

Check your understanding

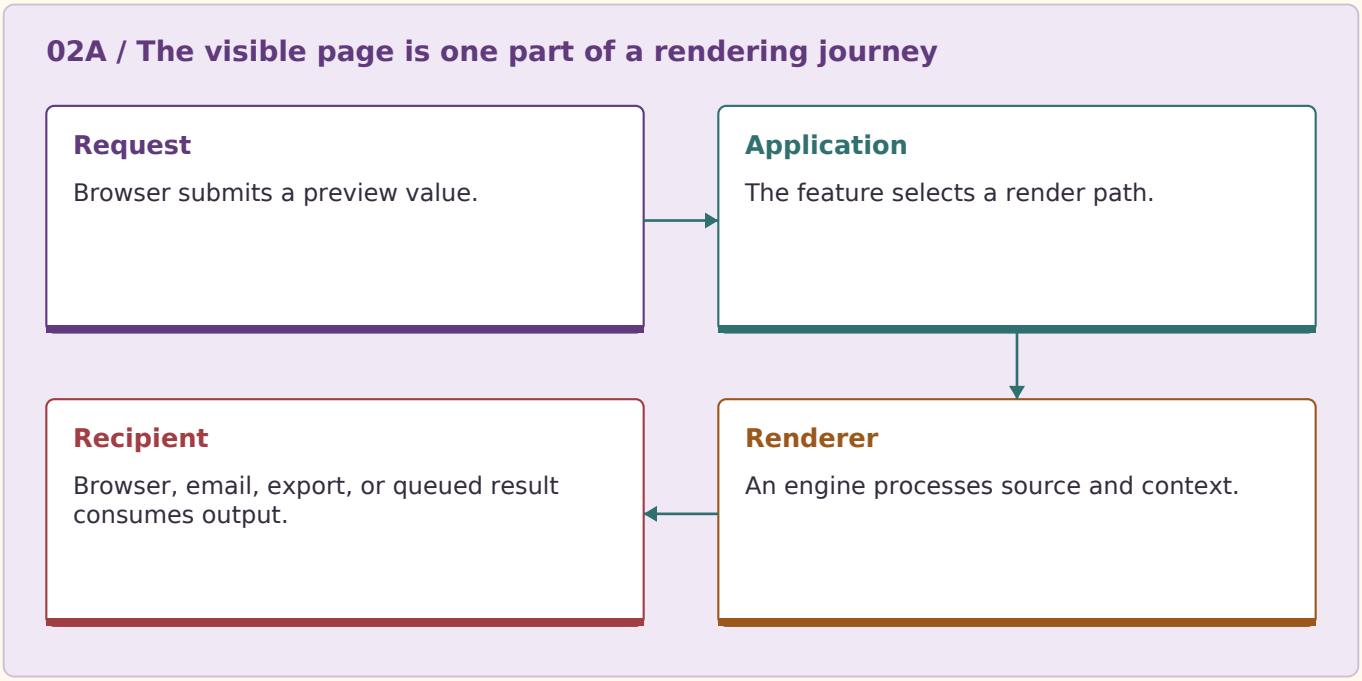
1.1 What does rendering mean?	**1.2** Are braces alone proof of SSTI?	**1.3** Why can identical input behave differently?

Answer explanations are in the full ebook. Sources: OWASP, JINJAAPI, JINJALANG

02. Follow a request to the machine that renders

Learn to: Identify browser, application server, template worker, and output recipient.

Think of it this way: A restaurant customer orders at the counter, the kitchen prepares the meal, and a delivery person takes it elsewhere. Watching the counter does not show every kitchen step.



Find the rendering stage before deciding what a changed page proves.

Browser and server

A browser is the program you use to open websites. A server is a program that receives requests and sends responses. A web application is the software that handles features such as login, search, checkout, and invoice generation. One computer can run several server programs.

HTTP is a set of rules for these requests and responses. A request includes a method, a target, headers, and sometimes a body. The method describes the kind of operation. The target identifies what is requested. Headers carry extra information. A body can contain submitted form values or JSON data.

The application may render a template while handling that request. It may instead save data for a background worker. That worker is another program or task which renders later. The finished result may go to your browser, an email inbox, a PDF file, or an administrator.

Server-side has a specific meaning

Server-side evaluation happens in the application's rendering environment before the browser receives its result. Client-side evaluation happens in code running in the browser. The same brace text can appear in both systems. Seeing a changed web page does not identify which system changed it.

First inspect the raw response in your browser's network tools or a replay tool. If a calculation is already present before page scripts run, that supports evaluation upstream of those scripts. It does not by itself identify the exact component: a gateway or another service may also transform responses. Map the path and use controlled comparisons.

Transport and trust

HTTPS protects the connection against certain network attackers. It does not decide whether a submitted name is safe template source. Login identifies a user under the application's authentication rules. It does not automatically grant permission to execute template instructions. These controls answer different questions.

In the local lab, your browser sends JSON to a loopback application. The application starts a short-lived renderer. A separate local receiver records a fixed callback exercise. These are teaching roles, not internet targets. The lab displays which step produced each observation.

Build a request map

For a preview feature, note where data enters, whether it is stored, which process renders it, and who receives the result. A blank response can still precede a successful email render. A cache can show an older result. This map prevents tests against the wrong stage and helps explain why a request and its effect can be far apart.

Key takeaway. Find the rendering stage before deciding what a changed page proves.

Check your understanding

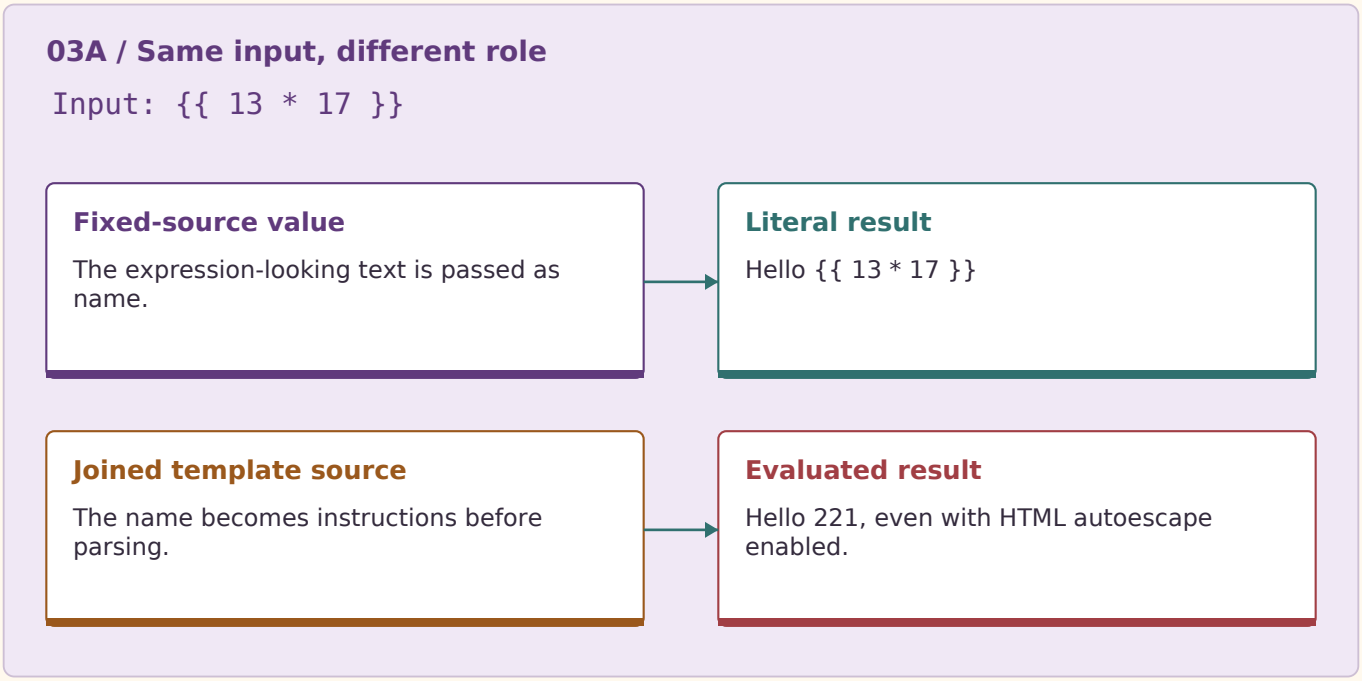
2.1 Does HTTPS prevent source injection?	**2.2** Why can an immediate response be blank?	**2.3** What helps separate browser evaluation from server rendering?

Answer explanations are in the full ebook. Sources: HTTP, HTML, ACADEMY

03. Escaping output does not protect template source

Learn to: Separate template parsing from HTML interpretation and choose the control for each boundary.

Think of it this way: A printer can safely package the finished letter while still obeying malicious instructions that were slipped into its print program.



Use fixed template source for SSTI prevention and context-appropriate output handling for later parsers.

Two parsers, two decisions

A parser reads text according to a language's rules. The template engine parses its source on the server. A browser later parses HTML output. A security control applied to one language does not automatically control the other.

Consider these two conceptual paths:

```
# Fixed source; the changing name is a value.
env.from_string("Hello {{ name }}").render(name=input_value)

# Changing input becomes source before rendering.
env.from_string("Hello " + input_value).render()
```

In a normal single Jinja render, the first path keeps brace text inside name as data. The second lets the template parser see the brace text as instructions. HTML autoescaping can be enabled in both paths; the second can still evaluate arithmetic.

What HTML escaping does

HTML escaping represents special HTML characters as text. For example, a less-than sign can become `<`. That helps prevent a displayed name from being interpreted as an HTML tag in a suitable output context. It does not remove the template engine's authority to evaluate source that has already been constructed.

SSTI and XSS can therefore appear together, separately, or neither. SSTI concerns untrusted instructions interpreted by a server template system. XSS concerns unwanted script execution in a browser. One rendered HTML tag is not proof of either server code execution or browser script execution.

Check the later consumer

A template may generate JavaScript, a URL, an email header, a shell configuration, or another template. Each later consumer has its own language and rules. Turning autoescape on does not make every generated output safe for every destination.

Likewise, marking a value safe for HTML does not mean it is approved template source. A safe marker is a claim about a particular output handling contract. Review who can make that claim and whether it matches the real consumer.

Practice the distinction

In the lab, compare a brace expression and a small HTML marker in data mode and source mode. The result panel displays output as text rather than inserting it as active HTML. This lets you inspect escaped characters without creating a browser attack. Keep three questions separate: did the engine evaluate, did the output contain markup, and did a later consumer execute anything?

Key takeaway. Use fixed template source for SSTI prevention and context-appropriate output handling for later parsers.

Check your understanding

3.1 Can autoescape be on while SSTI exists?	**3.2** Does HTML output prove XSS?	**3.3** Why is the lab result panel text-only?

Answer explanations are in the full ebook. Sources: JINJAAPI, MARKUP, HTML

Sources and further reading

Research cutoff: 1 November 2026. The catalog distinguishes consulted material from targeted further reading. It does not claim every linked page was audited end to end or every client was deployed. Live documents change. Historical fixes are not current release recommendations. Original diagrams and synthetic exercises do not imply vendor endorsement.

OWASP - OWASP SSTI prevention

Consulted targeted sections. Source/data separation and layered controls.

https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Template_Injection_Prevention_Cheat_Sheet.html

ACADEMY - PortSwigger SSTI learning topic

Consulted targeted sections. Source control, identification and capability analysis.

https://portswigger.net/web-security/server-side-template-injection

HTTP - RFC 9110 HTTP Semantics

Targeted further reading. Requests, responses, representations and method meaning.

https://www.rfc-editor.org/rfc/rfc9110

HTML - WHATWG HTML Living Standard

Targeted further reading. HTML interpretation happens after server rendering in the browser.

https://html.spec.whatwg.org/

JINJAAPI - Jinja API

Consulted targeted sections. Environment, loading, rendering and autoescape.

https://jinja.palletsprojects.com/en/stable/api/

JINJALANG - Jinja template language

Targeted further reading. Expressions, tags, filters, lookup and escaping.

https://jinja.palletsprojects.com/en/stable/templates/

MARKUP - MarkupSafe escaping

Targeted further reading. Markup flags describe HTML trust, not whether input may become template source.

https://markupsafe.palletsprojects.com/en/stable/escaping/